
DAYTONE MOOD ANALYSER

DEVELOPER & TECHNICAL DOCUMENTATION

System Overview: DayTone is a self-hosted, GDPR-compliant Flask web application designed for personal mood tracking, local sentiment analysis, and machine learning-based burnout prediction. The platform leverages local NLP models (NLTK VADER/DistilBERT) and on-device machine learning classifiers to predict burnout risk across 12 unique physiological and behavioral features. This technical specification documents the software architecture, database schemas, ML pipeline dynamics, security configurations, and operations guide.

Document Type:	System Architecture & Specifications
-----------------------	--------------------------------------

Project Version:	1.0.0
-------------------------	-------

Deployment Platform:	Render Cloud Infrastructure (Private Redis + PostgreSQL)
-----------------------------	--

Developer Contact:	MCA Semester 3 Mini Project Team
---------------------------	----------------------------------

1. System Architecture

DayTone follows a modular, layered architectural design with distinct presentation, business logic, and data access boundaries. It is designed to work fully locally for development and automatically scale to a containerized service stack in production.

A. Presentation Layer

Responsible for handling client-side interactions, assets loading, and visuals:

- **User Interface:** Responsive HTML5/CSS3 templates styled with a dark theme, integrated CSS grids, and mobile-responsive viewport bindings.
- **WebGL Shaders (Orb):** A customized Three.js implementation that deforms a 3D sphere vertex mesh dynamically based on the user's daily metrics (mood, sleep, stress) using simplex noise equations.
- **Visual Charts:** Interactive dashboard graphics rendered client-side via Chart.js, visualizing history and goal milestones.
- **PWA Offline Caching:** A standard Service Worker (sw.js) intercepts fetch requests and handles caching for offline app execution.

B. Business Logic Layer (Flask Core)

The core Flask application manages authentication, data sanitization, logging schedules, and analytical workflows:

- **Blueprints:** Modular application structures mapping to distinct areas: Auth (users/GDPR), Mood (check-ins/charts/exports), and Admin (auditing/user management).
- **Local NLP engine:** Textual analyses run directly in app memory using NLTK VADER or DistilBERT models, preventing external data leaks.
- **ML Classifier Pipeline:** Scikit-learn predictor loading trained decision tree weights and classifying daily inputs into burnout risk categories.

C. Data & Infrastructure Layer

Manages storage state, cache stores, and container operations:

- **Relational DB:** Dual-target storage schema supporting SQLite (development) and PostgreSQL (production) accessed via SQLAlchemy.
- **Cache & Limit Store:** Private Redis instance used as a session cache and rate limit registry (Flask-Limiter).
- **Encryption Engine:** AES-256 Fernet symmetric encryption layers that secure sensitive text column inputs at rest.

2. Database Models & Schema Design

DayTone employs a highly relational database architecture. Below is the specification breakdown of the core models defined in `app/models.py`:

Model Name	Core Fields	Purpose & Relational Constraints
User	id, name, email, password_hash, role, organization_id, deleted_at	Core account data. Integrates soft-deletes (<code>deleted_at</code>). Relates to <code>UserProfile</code> , <code>MoodLog</code> , and <code>Goal</code> via cascading deletes.
UserProfile	id, user_id, age, gender, preferred_activity, calm_mode, predict_burnout	User preferences. <code>calm_mode</code> disables WebGL deformations for accessibility. One-to-one mapping with <code>User</code> .
MoodLog	id, user_id, log_date, mood_score, sleep_hours, stress_level, social_interaction, notes, sentiment_score, burnout_risk, deleted_at	Stores daily check-ins. notes are AES-encrypted at rest. Sentiment and burnout features are calculated upon save.
BurnoutHistory	id, user_id, log_id, prediction, confidence, algorithm_used, predicted_at, is_accurate	Operational model tracking predictor performance and audit logs. Captures user feedback on accuracy.
Goal	id, user_id, target_type, target_value, start_date, end_date, completed	Tracks personal sleep, mood, or activity metrics. Supports dashboard progress visualizations.
AuditLog	id, admin_id, action, target_type, target_id, detail, ip_address, performed_at	Admin activity log for SOC-2 accountability, mapping critical actions like role toggles and user soft-deletes.
Organization	id, name, invite_code	Multi-tenancy grouping. Users joining an organization share metadata boundaries.

A. Cryptographic Column Security (Notes Field)

DayTone does not store raw journals in plaintext. The database field `_notes_encrypted` is mapped via a custom SQLAlchemy hybrid property. Writing to `log.notes` encrypts the text automatically using AES-256 (via the cryptography Fernet module) with the `ENCRYPTION_KEY` environment variable. Decryption occurs only in app memory when reading from the getter.

3. Local NLP & Machine Learning Pipelines

A. Local Natural Language Processing (NLP)

Journal notes undergo localized sentiment parsing to extract emotional cues without compromising privacy:

- **NLTK VADER:** Utilized by default. It processes word-level emotional polarization vectors, calculating a normalized compound sentiment score.
- **DistilBERT (Optional Transformer):** If the application detects the transformers library and sufficient memory, it attempts to load a lightweight distilbert-base-uncased-finetuned-sst-2-english model. The resulting classifier outputs are scaled to compile sentiment compound scores.
- **Degraded Execution Mode:** If both engines fail to load, the system degrades to output a neutral 0.0 sentiment score, maintaining application availability.

B. Burnout Predictor Feature Matrix

The Machine Learning classifier evaluates 12 distinct mathematical features compiled at check-in time:

Feature Name	Data Type	Derivation / Source
mood_score	Integer (1-5)	User daily mood check-in selection.
sleep_hours	Float	User sleep duration input.
stress_level	Integer (1-5)	User daily stress rating input.
activity_done	Integer (0/1)	Boolean check for daily exercise.
social_interaction	Integer (1-3)	Low, Medium, or High social activity.
sentiment_score	Float (-1 to 1)	VADER sentiment score from journal notes.
avg_mood_7d	Float	Rolling 7-day average of user's mood scores.
avg_stress_7d	Float	Rolling 7-day average of stress inputs.
avg_sleep_7d	Float	Rolling 7-day average of sleep hours.
consecutive_bad_days	Integer	Count of consecutive days with mood score ≤ 2.
mood_variability	Float	Standard deviation of mood scores over 7 days.
is_weekend	Integer (0/1)	1 if log date is Saturday/Sunday, else 0.

C. Model Operations & Retraining (MLOps)

DayTone includes scripts to automate the data pipeline: scripts/export_db_to_training.py compiles historical database records into training matrices. The training script (train.py) fits models (Decision Tree, Logistic Regression, Random Forest), validates accuracy, and outputs a model.pkl payload containing the model weights and metadata. Drift checks (KS-test) and bias audits evaluate operational metrics on the Admin page.

4. Security, PWA, & Accessibility Specifications

A. Security Controls & GDPR Hardening

DayTone implements key application security and compliance standards:

- **HTTP Security Headers:** Talisman enforces strict content security policies (CSP) including frame-ancestors 'self' and form-action 'self'. In production, sessions rely on secure, HTTP-only cookies (SESSION_COOKIE_SECURE=True).
- **Brute-Force Protection:** Auth views are protected via flask-limiter. In production, limits are stored in a private Redis cache using token bucket algorithms.
- **Admin Tokenization:** Direct registrations are disabled. Admins generate cryptographically secure database-backed invite tokens to register other staff, avoiding configuration credentials.
- **Right to Be Forgotten:** Account purge functions wipe profiles and related DB entries instantly.
- **Crisis Resource Reveal Shield:** If a High burnout risk is predicted, emergency lifelines are rendered on the UI, covered by an opt-in reveal shield to prevent immediate user anxiety.

B. Progressive Web App (PWA) Dynamics

DayTone operates as a modern Progressive Web App (PWA):

- **Service Worker caching:** The application installs sw.js. The worker cache stores assets locally. If the user loses connectivity, they receive cached views and a graceful offline.html fallback page.
- **Cross-Tab Broadcast Sync:** Utilizes the browser's BroadcastChannel. When a user submits a log in one tab, the update broadcasts to other open tabs, triggering a toast alert.
- **Query Caching:** Frequently accessed endpoints (such as goals lists and daily heatmaps) are cached via Flask-Caching, with automatic cache invalidation upon check-in updates.

C. Accessibility & Motion Controls (WCAG 2.1)

The dashboard includes accessibility features to support diverse user needs:

- **Calm Mode:** A profile setting that disables WebGL vertex simplex deformations on the 3D Orb and stops Chart.js animation loops, supporting users with motion sensitivities.
- **A11y Landmarks:** Implements standard skip-to-content links, clear tab-focus rings, layout landmarks (<main>, <header>, <footer>), and explicit aria-live regions.

5. Operations & Developer Workflows

A. Local Development Setup

To configure a local developer environment, execute the following commands in order:

```
# 1. Initialize environment and dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt

# 2. Configure environment settings
cp .env.example .env
# Open .env and set SECRET_KEY, ENCRYPTION_KEY, etc.

# 3. Run database migrations
flask db upgrade

# 4. Generate training baseline data & train model
python -m app.ml.generate_data
python -m app.ml.train

# 5. Run local development server
python run.py
```

B. Test Execution & Analysis

To verify application features, execute styling validation and unit tests:

```
# Run the PyTest automation suite
pytest -v

# Verify PEP-8 coding compliance
flake8 app/

# Run metrics drift diagnostics
python scripts/monitor_drift.py

# Run model bias audits
python scripts/bias_audit.py
```

C. Production Deployment (Render Blueprint)

Production deployments utilize the IaC configuration in render.yaml. The build command runs flask db upgrade, while the start command runs a gunicorn cluster: gunicorn "app:create_app()" --workers 2 --threads 2 --bind 0.0.0.0:\$PORT --preload. A private Redis cluster stores sessions and rate limit logs within Render's internal subnet.